



DATA STRUCTURES

CONTENTS



34. Minimum Spanning Trees-I

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the Graph ADT and Graph Traversals BFS and DFS. In this module we will discuss about Minimum Spanning Trees.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the concept of Spanning Trees
- To discuss Minimum Spanning Trees and its properties
- To explain the Greedy approach to finding Minimum Spanning Trees

34.1 Spanning Trees

Suppose you have a connected undirected graph. As you can recall connected means every node is reachable from every other node and undirected means edges

of the graph do not have an associated direction. A spanning tree of the graph is a connected sub-graph in which there are no cycles .

A spanning tree of a graph is just a sub-graph that contains all the vertices and is a tree. A graph may have many spanning trees. Spanning trees of the complete graph with 4 vertices is given in Figure 34.1. A spanning tree has the fewest number of edges possible while still retaining a connection between all the vertices in the component. If the component contains n vertices, the spanning tree contains $n - 1$ edges. When you traverse all the vertices of an undirected graph, you generate a spanning forest.

To find a spanning tree of a graph, we must pick an initial node and call it part of the spanning tree. Then we do a search (either BFS or DFS) from the initial node and each time you find a node that is not in the spanning tree, add both the new node *and* the edge you followed to get to it to the spanning tree

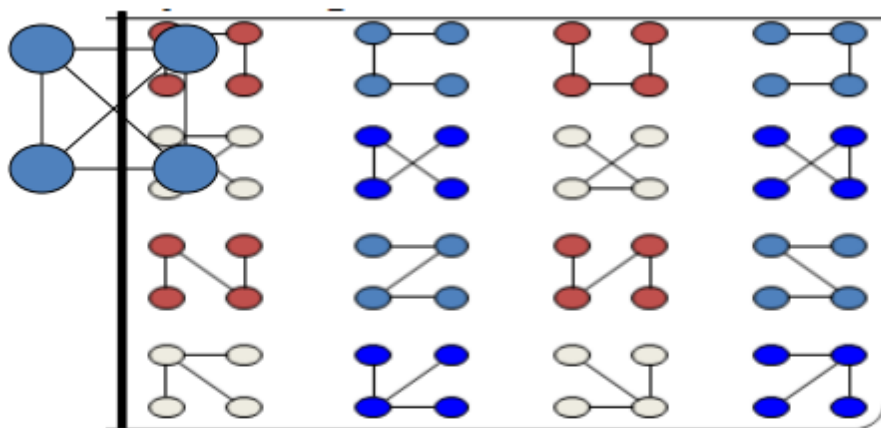


Figure 34.1 Spanning Trees of the complete Graph of 4 Vertices

34.2 Minimum Spanning Trees

In a weighted graph, the weight of a sub -graph is the sum of the weights of the edges in the sub-graph. A minimum spanning tree (MST) for a weighted undirected graph is a spanning tree with minimum weight.

The Minimum Spanning Tree for a given graph is the Spanning Tree of minimum

cost for that graph as shown below:



Figure 34.2 Minimum Spanning Tree

4.3 Creating a Spanning Tree

Either DFS or BFS can be used to create a spanning tree. When DFS is used, the resulting spanning tree is known as a depth first spanning tree. When BFS is used, the resulting spanning tree is known as a breadth first spanning tree. Adding a non-tree edge into any spanning tree, will create a cycle.

- Assume you have an undirected graph, $G = (V, E)$
- Spanning tree of graph G is tree

$$T = (V, E_T \subseteq E)$$

- Tree has same set of nodes
- All tree edges are graph edges

For a given tree T in a graph G , the edges and vertices of T are called tree edges and tree vertices, and the edges and vertices of G that are not in T are called non-tree edges and non-tree vertices.

$$E(G) = T(\text{tree edges}) + N(\text{nontree edges})$$

where T is the set of edges E_T used during search and N is the set of remaining edges

34.4 Properties of Spanning Trees

34.4.1 Property 1 of Spanning Trees

Given a graph $G = (V, E)$, and spanning tree: $T = (V, E_T)$ the property states that for any edge c in G but not in T , there is a simple cycle containing only edge c and edges in spanning tree. This is shown in Figure 34.3.

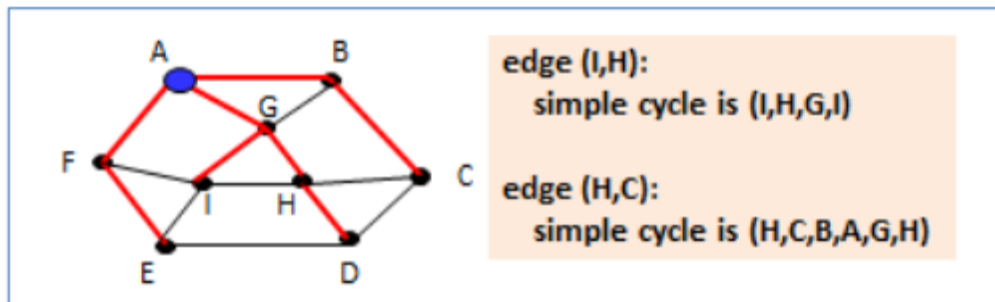


Figure 34.3 Spanning Tree –Property 1

34.4.2 Property 2 of Spanning Tree

Given a graph $G = (V, E)$, and spanning tree: $T = (V, E_T)$ the property states that for any edge c in G but not in T , there is a simple cycle Y containing only edge c and edges in spanning tree. Moreover, inserting edge c into T and deleting any edge in Y gives another spanning tree T' (Figure 34.4).

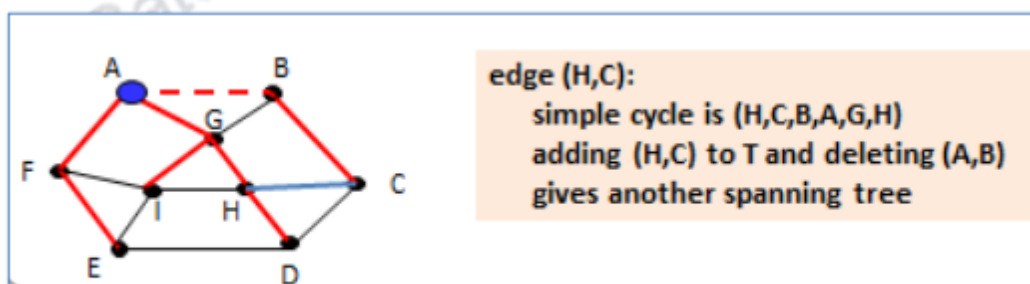


Figure 34.4 Spanning Tree-Property 2

34.5 Building BFS/DFS Spanning Trees

Before we build the spanning tree let us define the concept of frontier edge. A frontier edge for a given tree T in a graph is a non-tree edge with one endpoint in T , called its tree endpoint, and one endpoint not in T , its non-tree endpoint.

Let T be a tree subgraph of a graph G , and let S be the set of frontier edges for T . The function $\text{nextEdge}(G,S)$ chooses and returns as its value the frontier edge in S that is to be added to tree T . After a frontier edge is added to the current tree, the procedure $\text{updateFrontier}(G,S)$ removes from S those edges that are no longer frontier edges and adds to S those that have become frontier edges. Figure 34.5 gives a generic tree growing algorithm.

Algorithm: Tree-Growing(G, v)

Input: a connected graph G , a starting vertex $v \in VG$, and a selection-function nextEdge .

Output: an ordered spanning tree T of G with root v .

Initialize tree T as vertex v .

Initialize S as the set of proper edges incident on v .

While $S \neq \emptyset$

Let $e = \text{nextEdge}(G, S)$.

Let w be the non-tree endpoint of edge e .

Add edge e and vertex w to tree T .

$\text{updateFrontier}(G,S)$.

Return tree T .

Figure 34.5 Generic Tree Growing Algorithm

35.5.1 BFS and DFS as Tree-Growing

The depth-first and breadth-first searches use opposite versions of nextEdge. The nextEdge version of breadth first search selects a frontier edge whose tree endpoint was discovered earliest (least recently). The nextEdge version of depth first search selects a frontier edge whose tree endpoint have been most recently discovered. Let us consider the example shown in Figure 34.6.

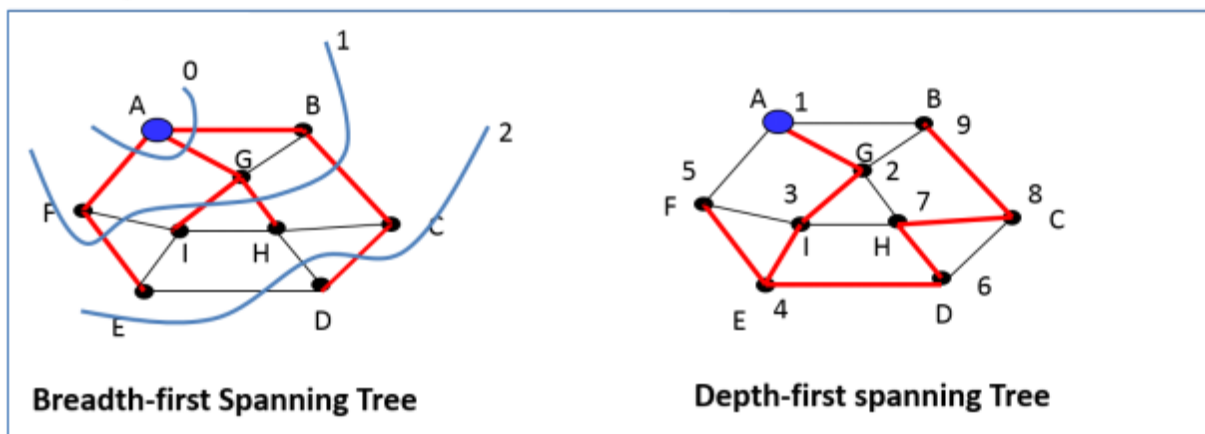


Figure 34.6 BFS/DFS Spanning Trees

Now the graph is shown and we first create the spanning tree using Breadth First search.

34.5.1 Example: BFS for Spanning Tree Construction

BFS uses the queue data structure for maintaining the yet to be discovered edges. Initially the edge from dummy node to A is inserted in the queue. Now since this is the only entry in the queue it is deleted and output as (dummy,A). Now we get all edges from A [(A,B),(A,G),(A,F)] are added to the queue as shown below and its destination nodes B,G,F are marked as done. These edges are added to the spanning tree. Now the first edge (A,B) from the queue is deleted and edges emanating from B [(B,C)] is inserted at the back of the queue but only if the destination node of edge C is not already done, and edge (B,G) is not added to the queue or the spanning tree. Edge (B,C) is added to the spanning tree. Now we again delete the edge from the front of the queue (A,G) and add the edges emanating from G [(G,I),(G,H)] to the

queue if the condition explained before is satisfied. These edges are also added to the spanning tree. Note that edge (G,B) is not inserted into the queue since node B has already been processed. Next the edge (A,F) is deleted from the queue and edges emanating from F and satisfying the condition discussed above (here **(F,E)**) is inserted at the back of the queue and added to the spanning .tree. Next edge (B,C) is deleted and edges from C here **(C,D)** is marked as done and added to the spanning tree. Now we have covered all the nodes of the graph and the process is complete.

```

[(dummy,A)]
  [(A,B),(A,G),(A,F)]
  [(A,G),(A,F),(B,C)].....
  [(A,F), (B,C), (G,I),(G,H)]
  [(B,C), (G,I),(G,H), (F,E)]
  [(G,I),(G,H),(F,E), (C,D)].....

```

34.5.2 Example: DFS for Spanning Tree Construction

DFS uses the stack data structure and the details are shown below. Here edges are added to stack if edge is not marked or destination node of the edge not marked (Figure 34.7). Initially we start from A and push all edges of A (AB,AF,AG) onto stack and mark A. Now we pop top edge from stack (**AG**), mark edge AG and node G and push all edges of G onto stack if the destination node of the edge is not already marked (GB,GH,GI), note GA is not added since it is marked. Now we pop top edge from stack (**GI**), mark node I and add edges of I (IF, IH, IE) to stack. Now note that IG is not added since it is already marked. Now we pop top edge from stack (**IE**), mark node E and add edges of E (ED, EF) to stack. Now note that EI is not added since it is already marked. Now we pop top edge from stack (**EF**), mark node F and add no edges to stack since destination nodes of edges (FA,FI) are marked and edge FE is marked. Now we pop top edge from stack (**ED**), mark node D and add edges of D (DC, DH) to stack. Now note that DE is not added since it is already marked. Now we pop top edge from stack (**DH**), mark node H and add edge of H (HC) to stack. Now note that the edge HD is not added to stack since the edge is marked and edges (HG, HI) since their destination nodes are marked. Now we pop top edge from stack (**HC**), mark node C and add edge (CB) to stack. Now note that the edges (CH, CB) are not added to stack since the edge are marked and edge CD is not added since its destination node is marked. Now we pop top edge from stack (**CB**), mark node B and

no more edges to stack since all nodes in the graph have been marked. The process is now complete.

Stack	No	Edge	Node- marked	Comments
A	1			Mark A & Push in A's Edges
AB AF AG	2	AG	G	pop AG & mark G & push edges from G
AB AF GB GH GI	3	GI	I	pop GI & mark I & push edges from I
AB AF GH IH IF IE	4	IE	E	pop IE & mark E & push edges from E
AB AF GH IF ED EF	5	EF	F	pop EF & mark F & push edges from F
AB AF GH IF ED	6	ED	D	pop ED & mark D & push edges from D
AB AF GH IF DC DH	7	DH	H	pop DH & mark H & push edges from H
AB AF GH IF DC HC	8	HC	C	pop HC & mark C & push edges from C
AB AF GH IF DC CB	9	CB	B	Now we stop since all nodes have been marked

Figure 34.7 Use of Stack for BFS for Spanning Tree

34.6 Weighted Spanning Trees

Now let us assume that we have an undirected graph $G = (V, E)$ with weights on each edge. The Spanning tree of graph G is a tree $T = (V, E_T \subseteq E)$.

The tree has same set of nodes, all tree edges are graph edges and weight of spanning tree = sum of tree edge weights .

A Minimal Spanning Tree (MST) is any spanning tree whose weight is minimal. In general, a graph has several MST's. There are many applications of circuit-board routing, networking, etc. The MST is acyclic because it is a tree. It is **spanning** because it covers every vertex and it is **minimum** because it has minimum cost.

34.6.1 Minimum Spanning Tree

If we wish we can have an MST start at a specific node. However, if there are weighted edges and all weighted edges are unique (if all edges have distinct weights), only one MST will exist. In a weighted graph, you can sum the weights for all edges in a spanning tree and attempt to find a spanning tree that minimizes this sum. A generic MST algorithm maintains an acyclic subgraph F of the input graph G . F is a subgraph of the minimum spanning tree of G , and we can use several algorithms for finding a **minimum spanning tree** for a component. Repeated application to all the components in a graph yields a **minimum spanning forest**.

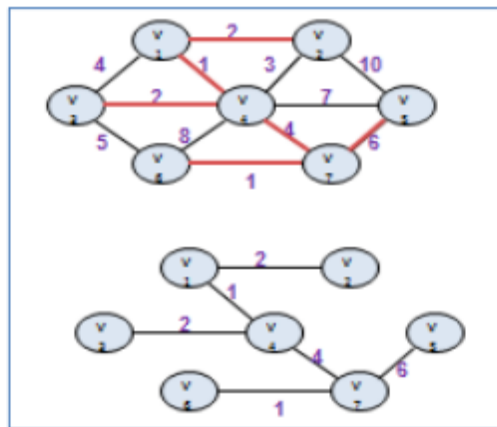


Figure 34.8 Example of Minimum Spanning Trees

36.2 Real Life Applications of MST

The standard application is to a problem like phone network design. You have a business with several offices; you want to lease phone lines to connect them up with each other; and the phone company charges different amounts of money to connect different pairs of cities. You want a set of lines that connects all your offices with a minimum total cost. It should be a spanning tree, since if a network isn't a tree you can always remove some edges and save. If it is constrained to bury the cable only along certain paths, then there would be a graph representing which nodes(cities) are connected by those paths. Some of those paths might be more expensive, because they are longer, or require the cable to be buried deeper; these

paths would be represented by edges with larger weights. A *minimum spanning tree* would be the network with the lowest total cost as given above (Figure 34.8).

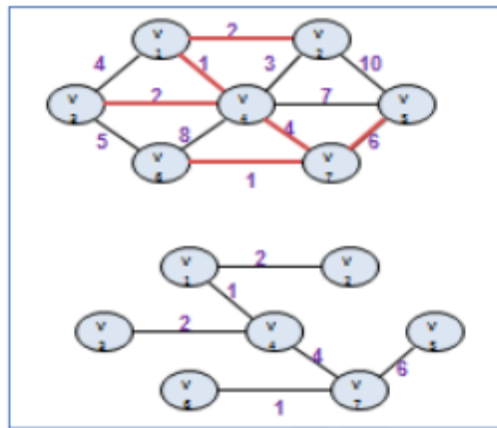


Figure 34.8 Example of Minimum Spanning Trees

36.2 Real Life Applications of MST The standard application is to a problem like phone network design. You have a business with several offices; you want to lease phone lines to connect them up with each other; and the phone company charges different amounts of money to connect different pairs of cities. You want a set of lines that connects all your offices with a minimum total cost. It should be a spanning tree, since if a network isn't a tree you can always remove some edges and save. If it is constrained to bury the cable only along certain paths, then there would be a graph representing which nodes(cities) are connected by those paths. Some of those paths might be more expensive, because they are longer, or require the cable to be buried deeper; these paths would be represented by edges with larger weights. A minimum spanning tree would be the network with the lowest total cost as given above (Figure 34.8).

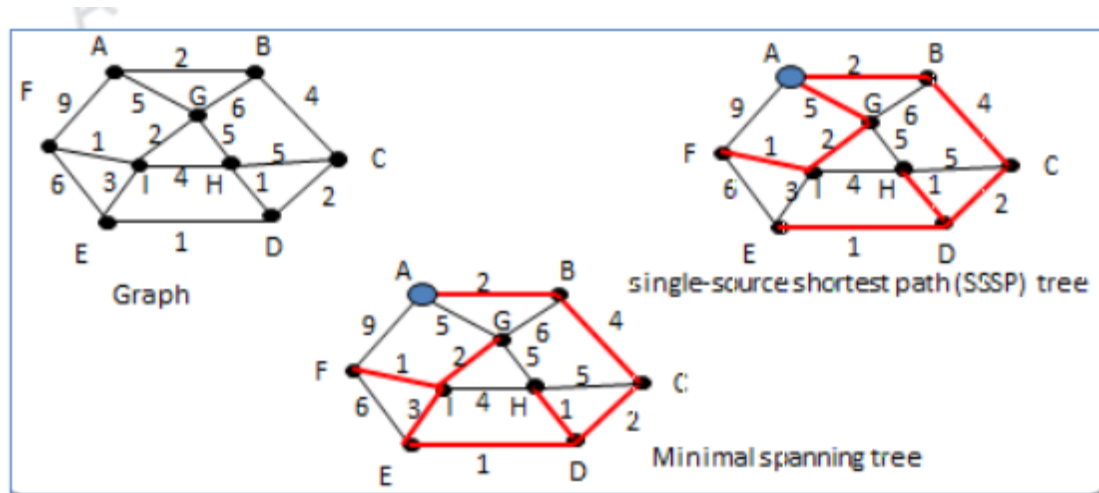


Figure 34.9 Minimal Spanning Tree

Another example could be the determination of how an airline can service all cities, while minimizing the total length of the routes it needs to support. In a weighted, undirected graph, it is a tree formed by connecting all of the vertices with minimal cost (Fig 34.9).

36.3 Single Source Shortest Path (SSSP) tree

This tree is constructed the input is a single source vertex in a weighted, directed graph and the job is to compute a shortest path for each possible destination. This is similar to BFS. Thus for a weighted graph $G = (V, E, w)$, the *single -source shortest paths* problem is to find the shortest paths from a vertex $w \in V$ to all other vertices in V . The Single Source Shortest Path (SSSP) has a given fixed start node. However in MST at any point in construction, we have a bunch of nodes that we have reached, and we look at the shortest distance from any one of those nodes to a new node (Figure 34.10).

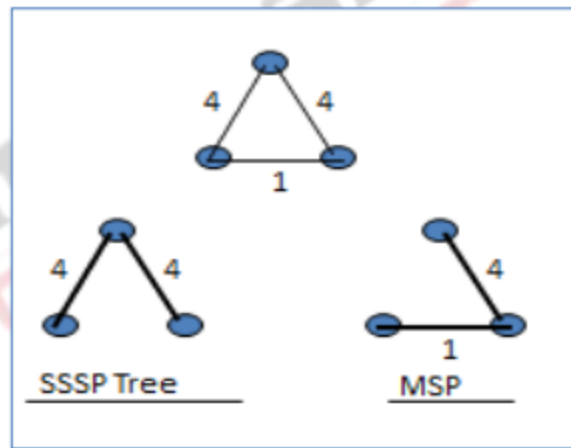


Figure 34.10 SSSP and MST

34.6.4 Property 3 of Minimal Spanning Trees

Given the Graph: $G = (V, E)$ and the Spanning tree: $T = (V, E_T)$. For any edge: c in G but not in T , there is a simple cycle Y containing only edge c and edges in spanning tree (already proved). Moreover, weight of c must be greater than or equal to weight of any edge in this cycle as given in Figure 34.11.

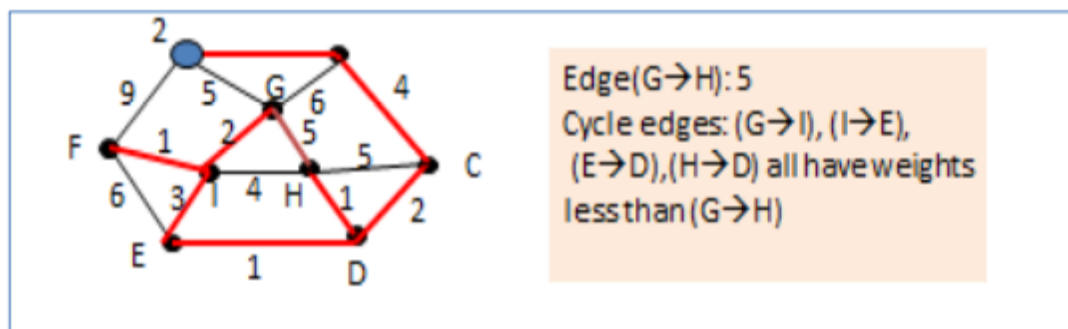


Figure 34.11 Minimal Spanning Tree-Property 3

34.7 MST algorithms

34.7.1 Borůvka's Algorithm

The first MST Algorithm was created by Otakar Borůvka in 1926. The algorithm was used to create efficient connections between the electricity network in the Czech

Republic. This algorithm is no longer used after Prim's and Kruskal's algorithms were discovered.

34.7.2 Greedy Algorithms for MST

Let us first understand what is meant by a greedy approach. Greedy algorithms apply to problems with *Optimal substructures* where optimal solutions contain optimal sub-solutions or has the *greedy choice property* where an optimal solution can be obtained by making the greedy choice at each step. Here we discuss the **greedy-choice property** where a globally optimal solution can be arrived at by making a locally optimal (greedy) choice.

A **greedy algorithm** always makes the choice for a locally optimal solution which seems the best at the current moment with the hope of arriving at a globally optimal solution. This approach efficiently computes a solution, works well for a wide range of problems, such as minimum-spanning-tree algorithms, Dijkstra's algorithm for shortest paths from a single source, etc. Greedy algorithms construct a solution to an optimization problem piece by piece through a sequence of choices that are feasible, locally optimal and irrevocable. The greedy approach to MST is an optimization problem where we need to find minimum cost for connecting all vertices. Let us understand the greedy approach (Figure 34.12). Let us assume that we have a set of disjoint components (initially nodes). Now we need to add the cheapest edge that joins disjoint components. We first start with node 1, and then choose the edge (1,2) since it is the cheapest edge associated with 1 and does not form a cycle. Now from 2, we choose edge (2,6) the cheapest edge from 2 without counting edge (2,1) (which will result in a cycle). Next we choose (6,3) and then (3,5) and finally (6,4) until we exhaust all the vertices.

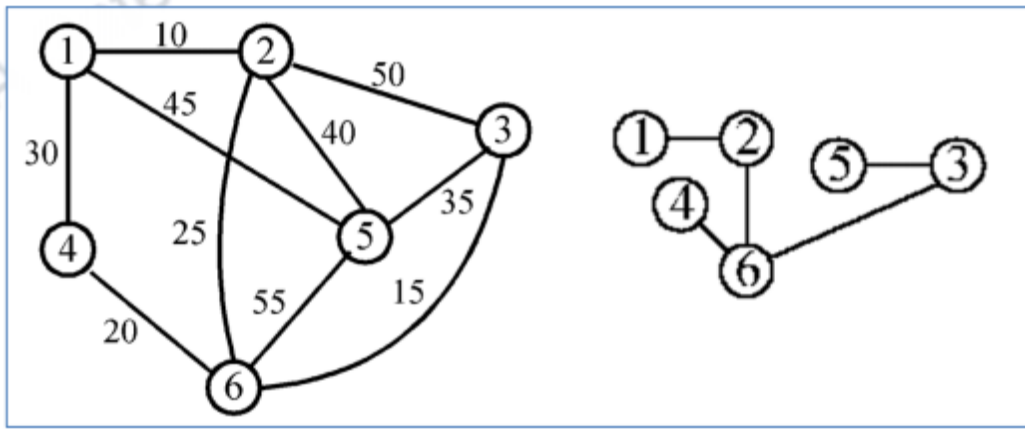


Figure 34.12 Minimal Spanning Tree-Greedy Algorithm

Both MST algorithms Kruskal, and Prim, determine an edge to be added to form the MST.

34.7.2.1 Greedy Algorithm 1 – Kruskal’s Algorithm

Kruskal’s algorithm grows a tree by repeatedly adding the least cost edge that does not introduce a cycle among the edges included so far. The intermediary solution is a spanning forest. Here we add the cheapest edges that join disjoint components . The Kruskal’s algorithm will be discussed in detail in Module 35.

34.7.2.2 Greedy Algorithm 2 – Prim’s Algorithm

Prim’s algorithm grows a single tree by repeatedly adding the least cost edge that connects a vertex in the existing tree to a vertex not in the existing tree . The intermediary solution is a sub-tree. Here we extend the tree by including the cheapest outgoing edge. The Prim’s algorithm will be discussed in detail in Module 36.

34.7.3 Why do the Greedy Algorithms work?

For simplicity, assume all edge costs are distinct. Let S be a subset of V , and suppose $e = (u, v)$ is the minimum cost edge of E , with u in S and v in $V-S$ and e is in every minimum spanning tree.

Summary

- Understood the concept of Spanning Trees
- Discussed Minimum Spanning Trees and its properties
- Explained the Greedy approach to finding Minimum Spanning Trees

you can view video on Minimum Spanning Trees-I



Web Links
www.cs.utoronto.ca/~tabrown/csc263/2014W/week9.ppt
home.cse.ust.hk/faculty/golin/COMP271Sp03/Notes/MyL08.pdf
www.cs.virginia.edu/~luebke/cs332/lecture25.ppt
www.cs.bilkent.edu.tr/~atat/502/DFS-TopologicalSort.ppt
https://www.cis.upenn.edu/~matuszek/cit594-2014/Lectures/28-spanning-trees.ppt
www.cs.cmu.edu/~roseh/Slides/cs211-graphs.ppt
www.cs.sjsu.edu/faculty/lee/cs157b/lanGravesMinimum%20Spanning%20Trees.ppt
delab.csd.auth.gr/~manolopo/graph/Lec9_MST.ppt
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014